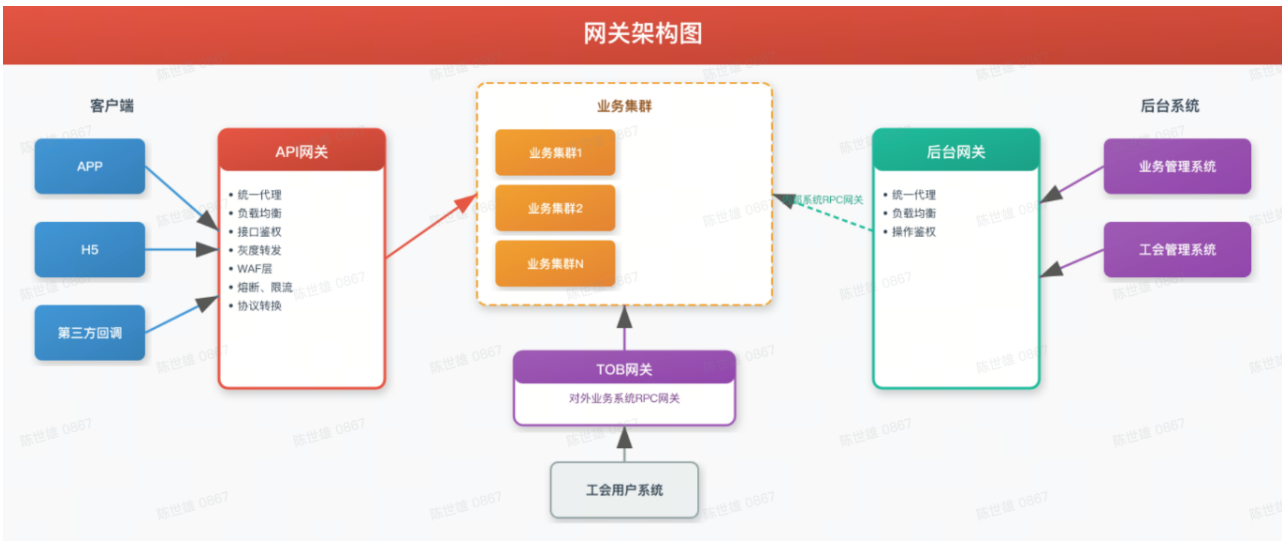
简介

目前公司对外流量的入口（除了个别日志埋点）

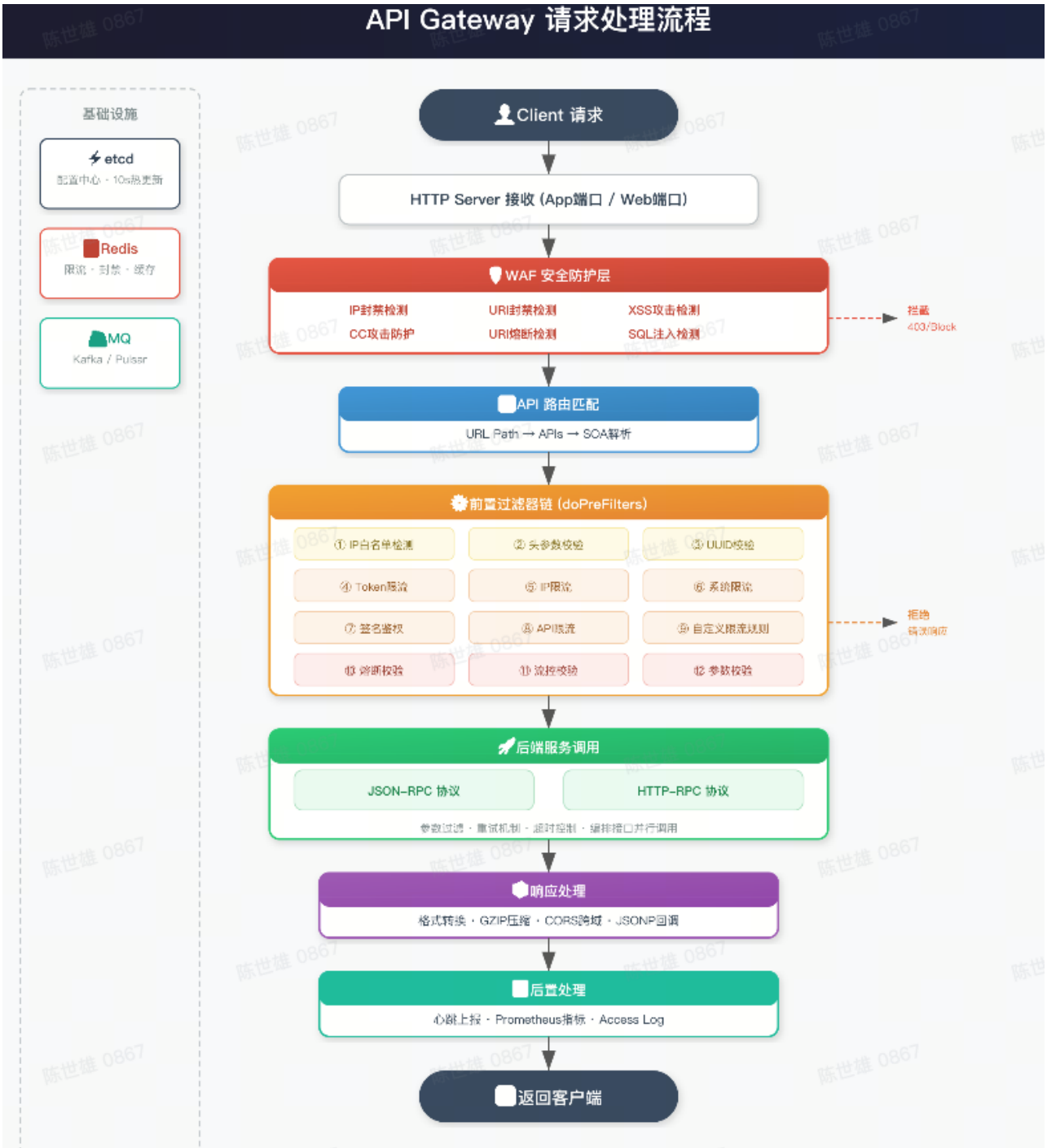
- 技术选型：golang,net/http, redis, pulsar, etcd
- 业务范围：他趣、timo、金融
- 分类： a.api网关 b. 后台网关 c. toB后台网关（给外部用的后台，不需要接入vpn，公会系统，代理商后台等）

架构图

整体架构：



API网关请求流程图



身份认证

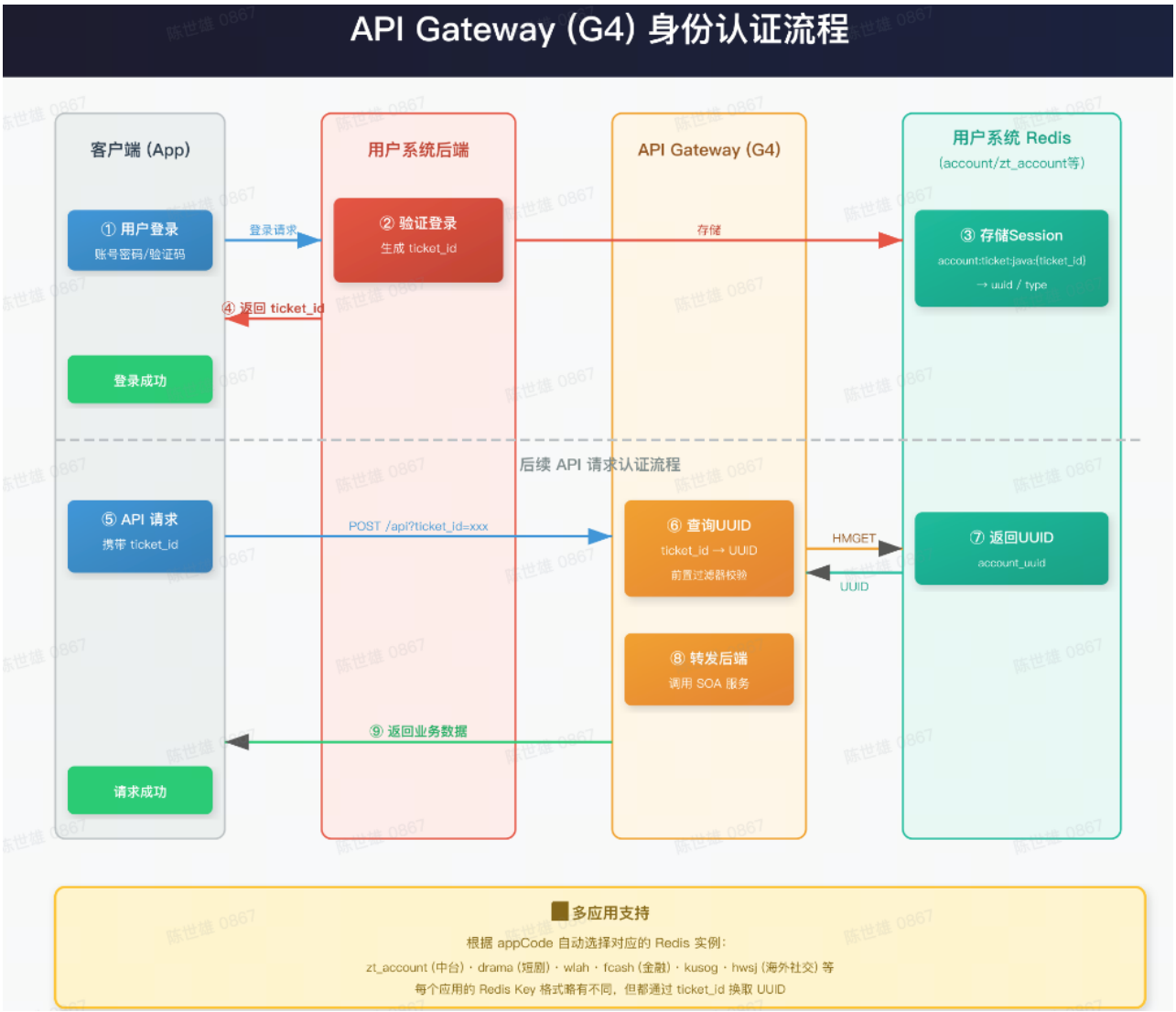
toB后台网关

对外系统的后台，没有统一的用户中心，各个服务各自为政了。公会系统，代理系统，都是自己写用户管理。

api网关

对外app等用户访问 流量大所以直接redis取缓存

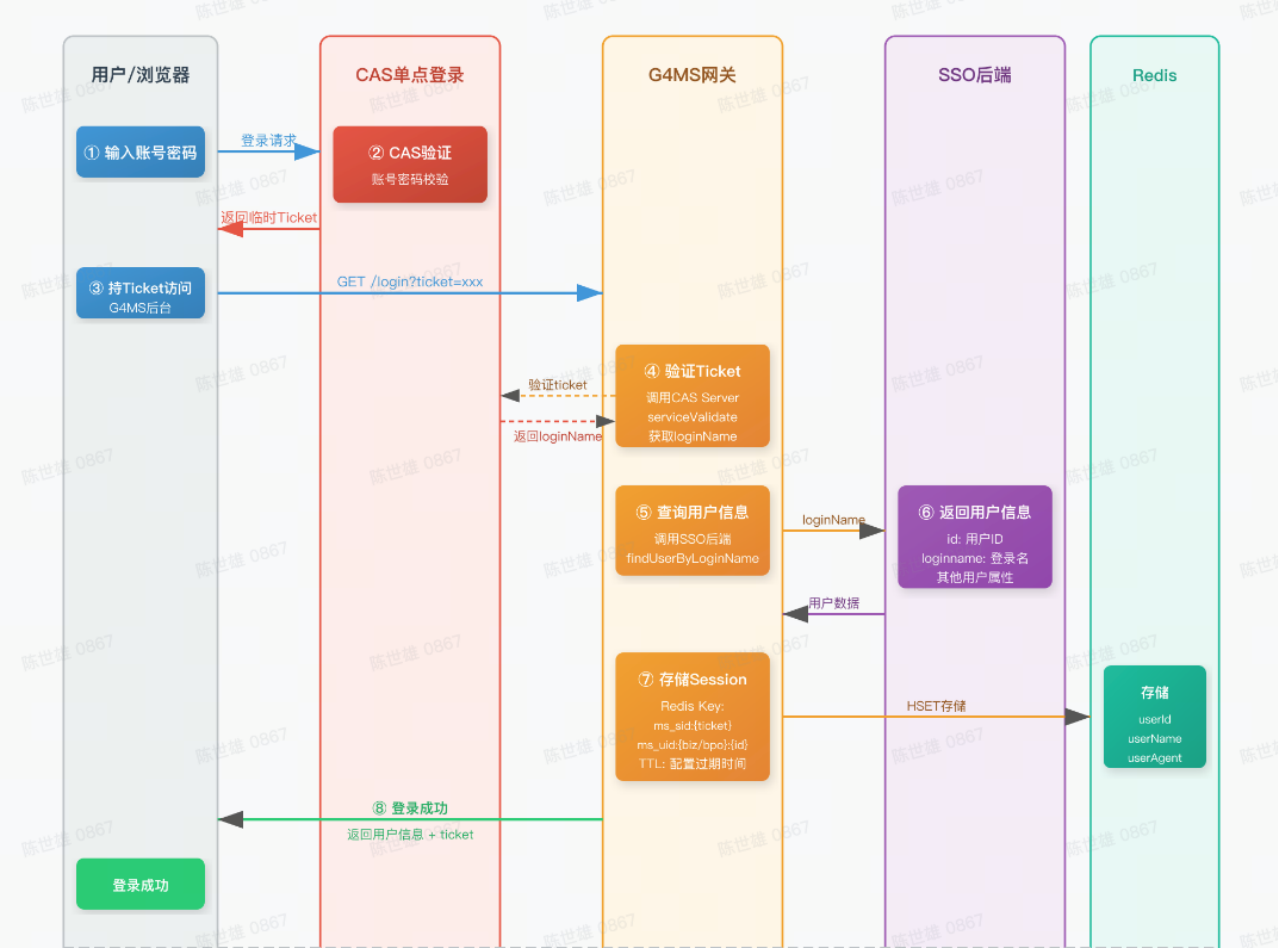
ticket_id可以进一步优化成jwt格式 返回用户动态密钥等

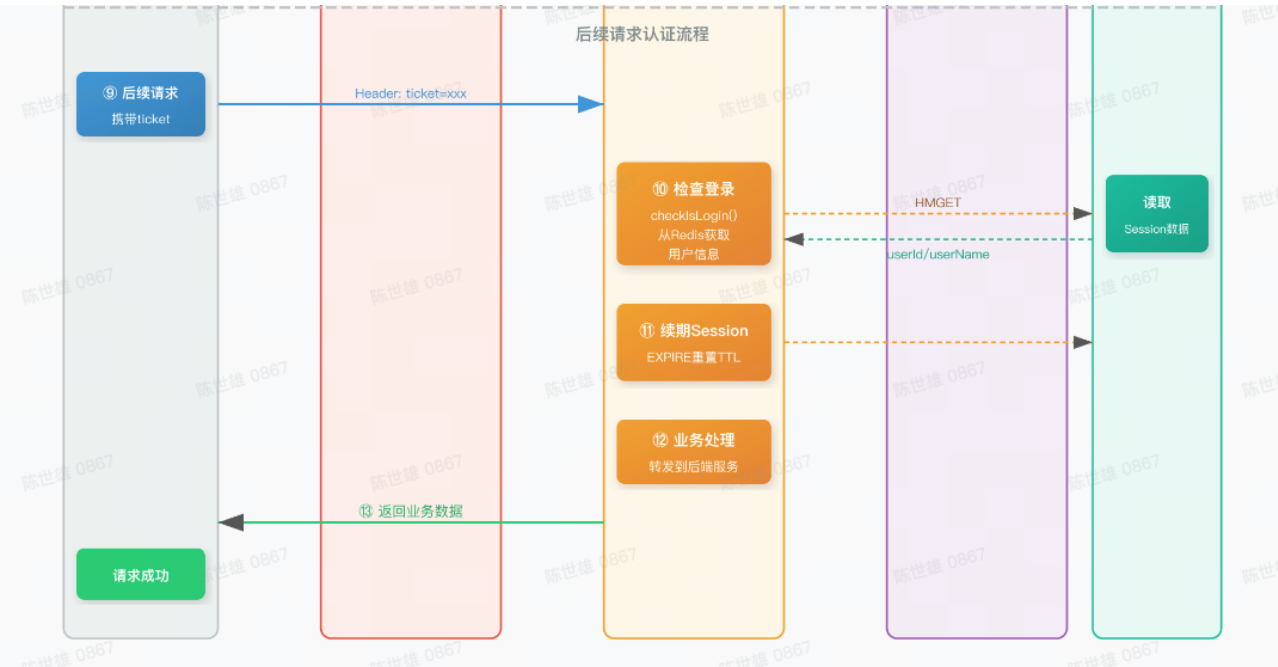


后台网关

内部后台统一cas维护 不是各自为政

cas单点登录系统 负责后台用户的ticket_id， g4ms验证ticket也是调用cas系统 然后去用户系统获取用户信息





api网关接口签名

签名用来验证前端api请求的合法性，不是恶意伪造的请求。

客户端签名计算

参与计算的字段：

- secretKey：客户端秘钥（写死，这里有风险 优化成不同用户跟随ticket下发）
- xkey, hkey, ikey, okey, tkey：五个是随机字符串（可以是多个），利用随机方式计算动态key
- time：终端时间戳
- token：设备id

加密过程：

1. 客户端按规则，随机生成：xkey, hkey, ikey, okey,tkey
2. 根据约定的计算公式，生成动态key
3. 客户端接口自身秘钥，结合动态key，时间戳，设备号，生产一个动态的加密结果
4. 客户端发起API请求，在header请求中添加相关加密值，加密结果

api网关核验签名

1. 获取客户端相关加密信息：
    a. xkey, hkey, ikey, okey, tkey
    b. time
    c. token
    d. 加密结果
2. 服务端利用上面信息，使用和客户端同样的公式计算动态key
3.服务端结合自身秘钥，结合动态key，时间戳，设备号，生产一个动态的加密结果
4.服务端对比自身计算的加密结果和客户端传递过来的是否一致，一致则表示合法的请求
5. 为了单次请求的信息被复用，服务端会对单次校验信息进行缓存校验，以防止重复利用。

存在问题

1. 客户端请求参数没有带进去计算签名，请求容易被拦截篡改。
2. 密钥和加密过程算法在android容易被人破解（实时上早已经被人破解了） 目前timo加了层防护，每个人的密钥不一样，登录后服务端下发。

请求参数和返回参数都通过对应密钥进行加密，增加一点破解难度。

流量管理

负载均衡

网关后台可以配置多个后端地址，将请求负载尽量均匀地分布到下游不同的服务节点上 不过目前都是只配置一个，配置k8s的service地址。。负载均衡实际上已经转移到k8s的的service能力了。

限流

目前用redis进行分布式流量控制，实现算法用最简单的计数器（key+过期时间） 配置多级流量控制，保护后端不被异常流量打挂

1. 接口全局限频（不管用户，key:userLimit:api）
2. 接口+token (key: api+token)
3. 业务系统+token （key: sysmte+token）
4. 用token级别（key: token） 如果redis挂了怎么办，，目前没有做降级本地计数器，如果redis挂了，直接默认通行。

参照oauth2.0不同媒体对于接口频控策略

（内存本地在redis挂了接手 使用滑动窗口计算比较合理 防止恶意ddos, 其实内部服务自己也需要做限流熔断降级）

重试

服务的波动。 目前后端超时时间默认5S，超时会进行重试。为了防止后端出现雪崩，目前每个系统重试有次数配置（比较大值，各个服务接口rpc请求之前默认3次重试），已满足大部分

协议转发

api网关请求： http://xxx.com/live_api/v1/getInfo?uuid=test

	旧版rpc	**http协议**
请方方式	post	post
uri	http://live-api/v1/Soa/jService?service=comet&method=getIp	http://live-api/v1/comet/getIp
body	form=WyJ0ZXN0Il0= （**ps**: ["test"]的base64编码）	{"uuid": "test"}
基本参数传递	query参数里面 ?distinctRequestId=h58r3g7yaqgk3tw6ovgkz36mmq9eq81f&timestamp=1773294659&env=online&lang=zh_cn&origin_system=fe-admp-pro-admin	header - -

返回值	{
	"data": {
	"ip": "123"
	},
	"msg": "err",
	"code":111
	}
		{
		"ip": "123"
		}
		httpStatus，判断是否失败

内部流量其实不值钱 直接使用http(可视化阅读方便)，就有json-rpc是公司内部定的rpc协议

服务编排

提供前端BFF的支持，由前端人员自助将后端多个服务援口聚合为一个API接口， 提高前端页面的响应速度。（本质使用go协程并发处理， 以前是php接口所以聚合还是有用）

目前实现依托golang轻量协程能力。每次请求，直接并发n个后端请求协程，等结果返回再进行重写数据拆装。

编辑接口

API　请求参数　自定义限流列表　熔断配置

deprecated (不建议使用，计划下线的接口)

* 请求方式　POST　GET

单设备请求频率　0 次 / 0 秒

全局请求频率　0 次 / 0 秒

* 请求URL　#1　/merge/v1/party/findTabGift

* SOA映射

#1　goods　v1/goodsGiftInfo/findTabGift

ticket_id　type

#2　party-play-engine　v1/themeActivityRV2/findThemeGift

ticket_id　type

响应处理

异常处理　异常都不忽略

数据拆包　SOA接口1　重新封装　tabGift

SOA接口2　重新封装　themeGift

Web访问　是　否

陈世雄 0867